UPDATED PROJECT REPORT

# ANPR Backend

## FastAPI + Database Integration

Updated files: app.py and anpr.db

| COMPONENT | CURRENT PURPOSE |
|---|---|
| app.py | FastAPI endpoints, plate normalization, owner lookup, role handling, membership-period check. |
| anpr.db | SQLite database storing users, vehicles, memberships, gate/parking records, events, payments, and notifications. |
| Separate test client | A test.py script may call the API over HTTP from another directory; it is not listed as a file in the current backend folder. |

*Prepared from the supplied FastAPI code and the current anpr.db schema.*

# 1. Executive Summary

This report documents the current ANPR backend implementation after the project moved away from the earlier Fiftyville demonstration database. The current service uses FastAPI and connects to anpr.db, an SQLite database designed around user accounts, vehicle ownership, memberships, parking facilities, gate operations, and access-event records.

The latest POST endpoint receives a license plate, normalizes it, finds its registered vehicle owner, and then applies role-based or membership-date logic. This is a meaningful change from the earlier implementation, which only searched bakery_security_logs and inserted a row into history. Those old table names are not part of the supplied ANPR database.

**Current behavior in one sentence**

POST / accepts a JSON plate, removes non-alphanumeric characters and uppercases it, looks up the matching vehicle owner, returns admin/private for those roles, otherwise checks the first membership time window and returns True or False.

The database contains 12 application tables. The current endpoint directly reads only three of them: vehicles, users, and memberships. Other tables already support broader application features but are not yet used by this endpoint.

# 2. Current Files and Project Scope

The current core backend files are:

```
project/
├── app.py
└── anpr.db
```

## 2.1 app.py

Contains the FastAPI application, the Pydantic request model, GET and POST routes, SQL queries, role branching, membership date parsing, and the normalized() helper.

## 2.2 anpr.db

The SQLite database opened by db = SQL("sqlite:///anpr.db"). Because this is a relative path, launch the server from the directory where this database is expected, or change the connection string to a path that is explicitly configured.

## 2.3 External test client

The team has used a separate test.py file in another directory. It can remain outside this folder because it communicates with the running server by HTTP. The actual contents of test.py were not supplied with this version, so this report describes the request/response contract rather than asserting the exact client implementation.

# 3. What Changed from the Earlier Implementation

| Earlier version | Current version |
|---|---|
| Connected to fiftyville.db. | Connects to anpr.db. |
| Searched bakery_security_logs for a plate. | Looks up vehicles.plate_normalized and retrieves the owner from users. |
| Inserted a row into history when a match existed. | Does not currently insert any row; it returns a role or a membership decision. |
| Returned True/false based on the plate count. | Returns admin or private for those roles, otherwise True/False based on membership dates. |
| Used a Flask route. | Uses FastAPI endpoint decorators and a Pydantic PlateRequest model. |

This is not merely a database filename change. The new database represents a more complete parking/access-control domain, and the application now evaluates ownership and membership rather than checking a log-table match.

# 4. High-Level Request Flow

```
Client sends POST / with JSON {"plate": " ab-123 "}
                |
                v
FastAPI validates PlateRequest(plate: str)
                |
                v
normalized() removes non-alphanumeric characters and uppercases
                |
                v
vehicles.plate_normalized -> owner_user_id -> users
                |
          Owner found?
          /        \
        No          Yes
        |            |
  return "False"   role admin/private?
                   /     \
                 Yes      No (member/other allowed role)
                 |        |
            return role    query memberships by user_id
                           |
                      compare current time
                           |
                      return "True"/"False"
```

All endpoint responses are plain text because the handlers declare response_class=PlainTextResponse. These are not JSON booleans: True and False are text strings, while admin and private are also text strings.

# 5. FastAPI Application and Request Model

## 5.1 Imports

```
from time import strftime, strptime, localtime, time
from cs50 import SQL
from fastapi import FastAPI
from fastapi.responses import PlainTextResponse
from pydantic import BaseModel
```

| Import | Purpose in this code |
|---|---|
| FastAPI | Creates the API application and registers endpoints. |
| PlainTextResponse | Sets endpoint responses to plain text. |
| BaseModel | Defines the validated JSON body structure. |
| SQL | Runs parameterized SQL against SQLite. |
| strptime, localtime, time | Parse membership timestamps and obtain the current local system time. |
| strftime | Imported but not used in the current code; it can be removed unless needed later. |

## 5.2 Application and database setup

```
app = FastAPI()
db = SQL("sqlite:///anpr.db")
```

The app object is the FastAPI application served by Uvicorn. The SQL object points to the new SQLite file. The connection path is relative; the program may fail to find the intended file if the working directory is unexpected.

### 5.3 Pydantic request model

```
class PlateRequest(BaseModel):
    plate: str
```

A POST request must contain a JSON body with a plate field whose value can be validated as a string, for example:

```
{"plate": " ab-123 "}
```

FastAPI parses the request body and validates it against PlateRequest before calling checking(). This validates the request shape and type, not whether the plate exists or whether the owner is allowed access.

## 6. GET Endpoint

```
@app.get("/", response_class=PlainTextResponse)
def index():
    return "Todo"
```

GET / returns the placeholder text Todo. It does not report the server health, query the database, or describe an access decision. This route can later become a health/status response if the team needs one.

## 7. POST Endpoint: Detailed Logic

### 7.1 Normalize the incoming plate

```
data.plate = normalized(data.plate)
```

The incoming value is cleaned before any SQL query. This is intended to make formatting variations such as spaces, hyphens, and punctuation irrelevant to matching, as long as the value stored in vehicles.plate_normalized follows the same convention.

### 7.2 Find the owner

```
check = db.execute(
    "SELECT * FROM users WHERE id = "
    "(SELECT owner_user_id AS n FROM vehicles WHERE plate_normalized = ?)",
    data.plate
)
```

The inner query finds the vehicle whose normalized plate matches and returns its owner_user_id. The outer query finds the corresponding user. The alias n is not used by the outer query and can be removed. Because vehicles.plate_normalized is UNIQUE in the schema, at most one vehicle can match the normalized plate.

If no vehicle matches, the scalar subquery returns no owner value and the outer query returns an empty result. In that case, if check evaluates to false and the endpoint returns the text False.

**More targeted query**

The endpoint only uses the owner id, role, and ideally account status. Selecting every users column also retrieves password_hash unnecessarily. A JOIN that selects only required columns would follow the principle of least data.

### 7.3 Handle administrative and private roles

```
if check[0]["role"] in ["admin", "private"]:
    return check[0]["role"]
```

If the owner role is admin or private, the endpoint returns that role as plain text immediately. The membership query is skipped for these roles. The code does not return a boolean for these role branches, so the client must know that these string responses have special meaning.

### 7.4 Look up a standard member membership

```
member = db.execute(
    "SELECT * FROM memberships WHERE user_id = ?",
```

```
    check[0]["id"]
)
```

For users whose role is not admin or private, the endpoint retrieves membership rows linked to the user. The memberships table contains a user_id foreign key and stores the plan and validity window.

### 7.5 Parse dates and check the validity window

```
member[0]["starts_at"] = strptime(
    member[0]["starts_at"], "%Y-%m-%d %H:%M:%S"
)
member[0]["expires_at"] = strptime(
    member[0]["expires_at"], "%Y-%m-%d %H:%M:%S"
)

if (member[0]["starts_at"] <= localtime(time()) and
        localtime(time()) <= member[0]["expires_at"]):
    return "True"
return "False"
```

strptime() parses the stored timestamp strings into time.struct_time values. localtime(time()) obtains the current time according to the server machine's local time zone. The code compares the start, current time, and expiry values and returns True if the current time falls within the interval; otherwise, it returns False.

The comparisons use inclusive boundaries: a request is accepted when the current value is equal to the start or expiry timestamp. Whether the expiry boundary should be inclusive is a business rule the team should define explicitly.

## 8. normalized() Function

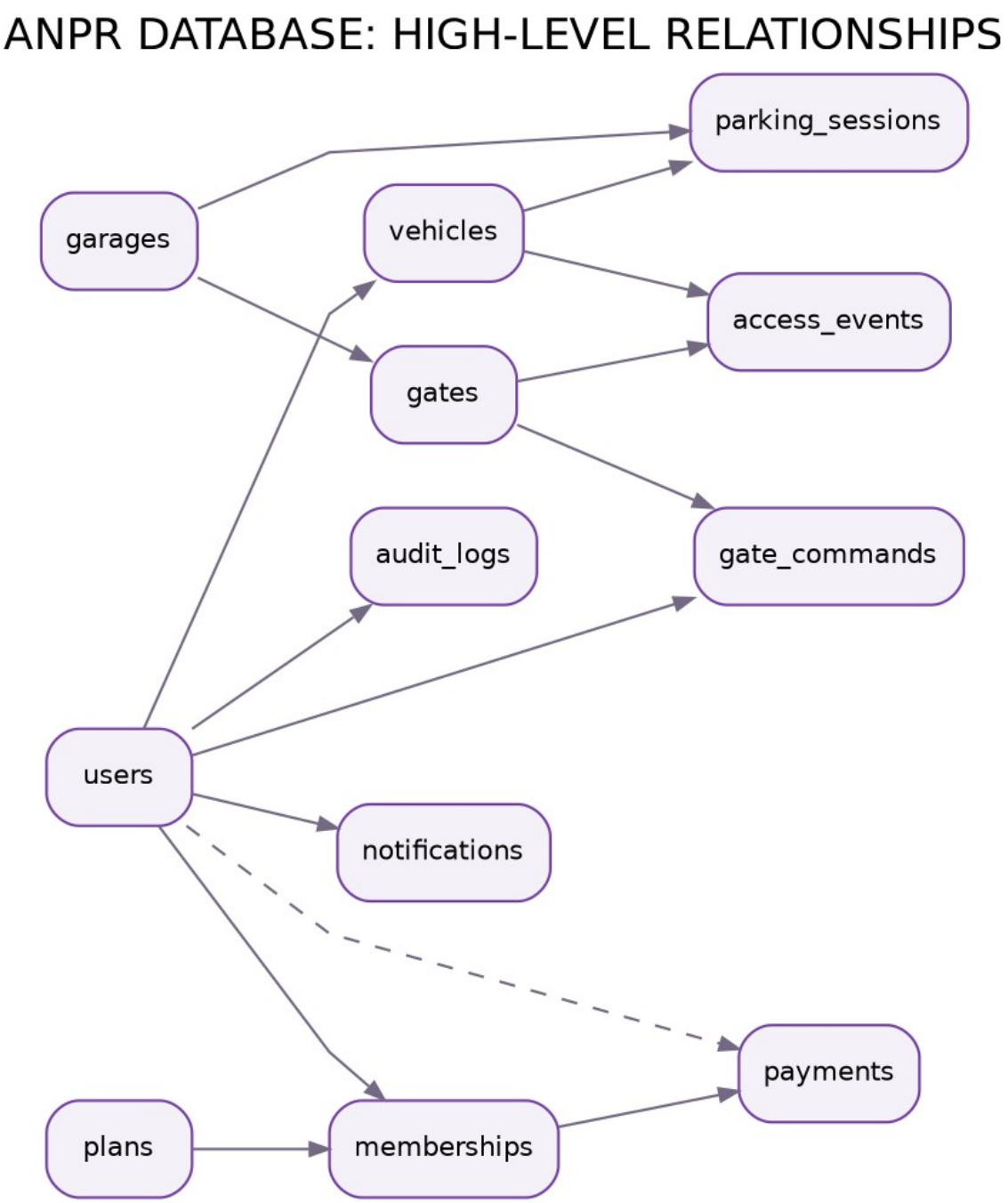
```
def normalized(plate: str) -> str:
    word = ""
    for i in plate:
        if i.isalnum():
            word += i
    return word.upper()
```

| Input | Normalized result |
|---|---|
| ab-123 | AB123 |
| ab 123 | AB123 |
| xy_789! | XY789 |

The helper walks through each character, keeps characters for which isalnum() is true, appends them to a result string, and converts the final string to uppercase. The database column plate_normalized should always be written using the same normalization rule. Also note that Python's isalnum() accepts Unicode letters and digits, not only ASCII A-Z and 0-9; constrain it if local plate rules require ASCII characters only.

## 9. Database Architecture Overview

The supplied anpr.db contains 12 application tables. The diagram below summarizes their main relationships. It omits individual columns to make the relationship structure easier to read.




## 9.1 Relationship summary

| Parent / source | Related table | Relationship meaning |
|---|---|---|
| users | vehicles | One user can own multiple vehicles; each vehicle has one owner. |
| users | memberships | A user can have multiple membership records. |
| plans | memberships | A plan can be used by many membership records. |
| memberships | payments | A membership can have payment records. |
| garages | gates | A garage can contain multiple gates. |

| Parent / source | Related table | Relationship meaning |
|---|---|---|
| vehicles and garages | parking_sessions | Each session refers to one vehicle and one garage. |
| gates and optional vehicles | access_events | A gate event may be associated with a recognized vehicle. |
| users and gates | gate_commands | A user requests a command for a target gate. |
| users | audit_logs / notifications | Users are linked to audit actions and notifications. |

# 10. Database Tables and Columns

The following table definitions are based on the supplied database structure. INTEGER, TEXT, and REAL are SQLite declared types. Constraints such as NOT NULL, UNIQUE, CHECK, and REFERENCES enforce important data rules when they are active on the connection and used correctly.

## 10.1 users

Stores user accounts and their global role/status.

| Column | Type | Key / constraint | Meaning |
|---|---|---|---|
| id | INTEGER | Primary key; auto-increment | Unique user identifier. |
| full_name | TEXT | NOT NULL | User display name. |
| email | TEXT | NOT NULL; UNIQUE | Login/contact email; duplicate values are prevented. |
| phone | TEXT | Nullable | Optional phone number. |
| password_hash | TEXT | NOT NULL | Password hash, not a plain-text password. |
| role | TEXT | NOT NULL; default 'member'; CHECK member/private/admin | Role used by the current endpoint to branch access logic. |
| status | TEXT | NOT NULL; default 'active'; CHECK active/suspended/deleted | Account lifecycle status; currently not checked by app.py. |
| created_at | TEXT | NOT NULL; default datetime('now') | Creation timestamp in SQLite datetime text format. |

## 10.2 vehicles

Stores registered vehicles and links each plate to an owner.

| Column | Type | Key / constraint | Meaning |
|---|---|---|---|
| id | INTEGER | Primary key; auto-increment | Unique vehicle identifier. |
| owner_user_id | INTEGER | NOT NULL; FK users.id | User account that owns the vehicle. |
| plate_normalized | TEXT | NOT NULL; UNIQUE | Canonical plate value used for matching; duplicates are prevented. |
| plate_raw | TEXT | Nullable | Original plate representation, if retained. |
| country | TEXT | Nullable | Optional country/registration context. |
| status | TEXT | NOT NULL; default 'active'; CHECK active/blocked | Vehicle status; currently not checked by app.py. |
| created_at | TEXT | NOT NULL; default datetime('now') | Vehicle registration timestamp. |

## 10.3 plans

Defines membership products/pricing.

| Column | Type | Key / constraint | Meaning |
|---|---|---|---|
| id | INTEGER | Primary key; auto-increment | Unique plan identifier. |
| name | TEXT | NOT NULL | Plan name. |
| duration_days | INTEGER | NOT NULL; CHECK > 0 | Plan duration in days. |
| price | INTEGER | NOT NULL; CHECK >= 0 | Price stored as an integer; the schema does not declare a currency here. |
| is_active | INTEGER | NOT NULL; default 1; CHECK 0 or 1 | Whether the plan is currently offered; not checked by app.py. |

## 10.4 memberships

Associates a user with a plan and a validity interval.

| Column | Type | Key / constraint | Meaning |
|---|---|---|---|
| id | INTEGER | Primary key; auto-increment | Unique membership identifier. |
| user_id | INTEGER | NOT NULL; FK users.id | Member account. |
| plan_id | INTEGER | NOT NULL; FK plans.id | Associated plan. |
| starts_at | TEXT | NOT NULL | Membership start timestamp; app.py parses as %Y-%m-%d %H:%M:%S. |
| expires_at | TEXT | NOT NULL; CHECK expires_at > starts_at | Membership expiry timestamp and interval consistency rule. |
| status | TEXT | NOT NULL; default 'active'; CHECK active/cancelled | Membership state; currently not checked by app.py. |
| created_at | TEXT | NOT NULL; default datetime('now') | Membership record creation timestamp. |

## 10.5 payments

Records payments related to memberships.

| Column | Type | Key / constraint | Meaning |
|---|---|---|---|
| id | INTEGER | Primary key; auto-increment | Unique payment identifier. |
| membership_id | INTEGER | NOT NULL; FK memberships.id | Membership being paid for. |
| amount | INTEGER | NOT NULL; CHECK >= 0 | Payment amount. |
| currency | TEXT | NOT NULL; default 'IQD' | Currency code. |
| method | TEXT | NOT NULL; default 'cash' | Payment method label. |
| paid_at | TEXT | NOT NULL; default datetime('now') | Payment timestamp. |
| recorded_by | INTEGER | Nullable; FK users.id | Optional user who recorded the payment. |

## 10.6 garages

Represents a parking facility.

| Column | Type | Key / constraint | Meaning |
|---|---|---|---|
| id | INTEGER | Primary key; auto-increment | Unique garage identifier. |
| name | TEXT | NOT NULL | Garage name. |
| capacity | INTEGER | NOT NULL; CHECK > 0 | Configured parking capacity. |

## 10.7 gates

Represents an entry/exit gate belonging to a garage.

| Column | Type | Key / constraint | Meaning |
|---|---|---|---|
| id | INTEGER | Primary key; auto-increment | Unique gate identifier. |
| garage_id | INTEGER | NOT NULL; FK garages.id | Parent garage. |
| name | TEXT | NOT NULL | Gate name. |
| direction | TEXT | NOT NULL; CHECK entry/exit | Whether the gate is an entry or exit gate. |
| last_seen | TEXT | Nullable | Optional last-seen/heartbeat timestamp. |

## 10.8 parking_sessions

Records a vehicle parking interval in a garage.

| Column | Type | Key / constraint | Meaning |
|---|---|---|---|
| id | INTEGER | Primary key; auto-increment | Unique parking-session identifier. |
| vehicle_id | INTEGER | NOT NULL; FK vehicles.id | Vehicle associated with the session. |
| garage_id | INTEGER | NOT NULL; FK garages.id | Garage where the session occurs. |
| entered_at | TEXT | NOT NULL; default datetime('now') | Entry time. |
| exited_at | TEXT | Nullable; CHECK NULL or >= entered_at | Exit time; NULL can represent an open session. |

## 10.9 access_events

Records vehicle observations and allow/deny decisions at gates.

| Column | Type | Key / constraint | Meaning |
|---|---|---|---|
| id | INTEGER | Primary key; auto-increment | Unique access-event identifier. |
| gate_id | INTEGER | NOT NULL; FK gates.id | Gate where the event occurred. |
| vehicle_id | INTEGER | Nullable; FK vehicles.id | Matched vehicle, when identification succeeds. |
| plate_read | TEXT | NOT NULL | Plate text detected by the vision/CV component. |
| confidence | REAL | Nullable | Optional recognition confidence score. |
| decision | TEXT | NOT NULL; CHECK allowed/denied | Access decision. |
| reason | TEXT | NOT NULL | Human-readable or machine-readable decision reason. |
| image_path | TEXT | Nullable | Optional path/reference to a captured image. |
| occurred_at | TEXT | NOT NULL; default datetime('now') | Event timestamp; indexed for time-based queries. |

## 10.10 gate_commands

Stores user-created commands for a physical gate.

| Column | Type | Key / constraint | Meaning |
|---|---|---|---|
| id | INTEGER | Primary key; auto-increment | Unique command identifier. |
| user_id | INTEGER | NOT NULL; FK users.id | User who requested the command. |
| gate_id | INTEGER | NOT NULL; FK gates.id | Target gate. |
| status | TEXT | NOT NULL; default 'pending'; CHECK pending/executed/expired/rejected | Command lifecycle state. |

| Column | Type | Key / constraint | Meaning |
|---|---|---|---|
| created_at | TEXT | NOT NULL; default datetime('now') | Command creation timestamp. |
| expires_at | TEXT | NOT NULL; CHECK expires_at > created_at | Expiration time. |

### 10.11 audit_logs

Records actions for accountability and investigation.

| Column | Type | Key / constraint | Meaning |
|---|---|---|---|
| id | INTEGER | Primary key; auto-increment | Unique audit record identifier. |
| user_id | INTEGER | NOT NULL; FK users.id | User who performed the action. |
| action | TEXT | NOT NULL | Action name. |
| target_type | TEXT | Nullable | Optional type of object affected. |
| target_id | INTEGER | Nullable | Optional identifier of the object affected. |
| details | TEXT | Nullable | Optional extra context. |
| occurred_at | TEXT | NOT NULL; default datetime('now') | Audit timestamp; indexed for time-based queries. |

### 10.12 notifications

Stores user-facing messages.

| Column | Type | Key / constraint | Meaning |
|---|---|---|---|
| id | INTEGER | Primary key; auto-increment | Unique notification identifier. |
| user_id | INTEGER | NOT NULL; FK users.id | Recipient user. |
| type | TEXT | NOT NULL | Notification category, e.g. expiring_soon or admin_message. |
| message | TEXT | NOT NULL | Notification body. |
| is_read | INTEGER | NOT NULL; default 0; CHECK 0 or 1 | Read/unread flag. |
| created_at | TEXT | NOT NULL; default datetime('now') | Creation timestamp. |

## 11. Indexes, Constraints, and Data Integrity

### 11.1 Important constraints

- users.email is UNIQUE, so two user rows cannot share the same email value.
- vehicles.plate_normalized is UNIQUE, so a normalized plate can identify at most one vehicle row.
- Foreign keys connect vehicles to users, memberships to users and plans, payments to memberships and optionally users, gates to garages, parking sessions to vehicles and garages, access events to gates and optionally vehicles, gate commands to users and gates, and audit_logs/notifications to users.
- CHECK constraints limit roles, statuses, gate directions, access decisions, nonnegative amounts, positive plan durations, positive capacities, and valid date ordering.
- The one_open_session partial unique index allows at most one parking session with exited_at IS NULL for each vehicle.

### 11.2 Indexes observed in the database

| Index | Table / column | Purpose |
|---|---|---|
| idx_events_time | access_events(occurred_at) | Supports time-based filtering and sorting of access events. |
| idx_audit_time | audit_logs(occurred_at) | Supports time-based audit log queries. |

| Index | Table / column | Purpose |
|---|---|---|
| one_open_session | parking_sessions(vehicle_id) WHERE exited_at IS NULL | Enforces at most one open session per vehicle. |
| SQLite auto-indexes | users.email; vehicles.plate_normalized | Automatically support UNIQUE constraints. |

### 11.3 Integrity snapshot of the supplied database

The supplied database passed SQLite PRAGMA integrity_check with result "ok". The foreign_key_check query returned 0 violation(s). The database currently contains 12 application tables (excluding SQLite internal tables).

| Table | Rows in supplied snapshot |
|---|---|
| users | 6 |
| vehicles | 6 |
| plans | 2 |
| memberships | 5 |
| payments | 2 |
| garages | 1 |
| gates | 2 |
| parking_sessions | 1 |
| access_events | 2 |
| gate_commands | 1 |
| audit_logs | 1 |
| notifications | 0 |

**These row counts describe the uploaded file at the time it was inspected; they will change as the application is used. No personal names, emails, password hashes, plate numbers, or other identifying row values are reproduced in this report.**

## 12. Which Database Tables the Current Endpoint Uses

| Table | Read / write in app.py? | How it is used |
|---|---|---|
| vehicles | Read | Finds the row whose plate_normalized matches the normalized request; supplies owner_user_id. |
| users | Read | Finds the owner and reads id and role. The current code does not validate status. |
| memberships | Read | Finds membership rows for a non-admin/non-private owner and checks the first row's time period. |
| plans | Not used | Has the plan associated with a membership; is_active is not checked. |
| payments | Not used | Payment records are not consulted before returning True. |
| access_events | Not used | The endpoint currently does not record the recognition result or decision here. |
| parking_sessions | Not used | The endpoint does not open/close a parking session. |
| garages, gates | Not used | No gate is selected and no physical gate operation is issued. |
| gate_commands | Not used | No command record is created by this endpoint. |
| audit_logs | Not used | No audit event is written by the current code. |
| notifications | Not used | No notification is created by the current code. |

**Important scope distinction**

The database schema supports access events, parking sessions, gate commands, audit logs, and notifications. The current app.py snippet does not yet use those features; it only looks up ownership and evaluates the role/membership rule.

# 13. Risks and Recommended Improvements

## 13.1 Handle an owner with no membership

If a user has a role other than admin or private and the memberships query returns an empty list, member[0] raises an IndexError. Check if member is empty before accessing member[0], and return the intended denial result or an explicit error response.

## 13.2 Check membership status and plan status

The current logic checks only starts_at and expires_at. A membership with status="cancelled" could still be accepted if its dates surround the current time. Likewise, plans.is_active is not checked. Add these checks if they are part of the intended access policy.

## 13.3 Handle multiple memberships deliberately

The query selects every membership for the user but only reads member[0]. Without ORDER BY, the first returned row is not a reliable way to select the newest or currently valid membership. Query for an eligible membership explicitly, or iterate across all membership rows and define a deterministic rule.

## 13.4 Check vehicle and user status

The schema supports vehicles.status values active/blocked and users.status values active/suspended/deleted, but the current query does not check either status. As written, a blocked vehicle or suspended/deleted user can still reach the role or membership branch. Confirm the team’s policy and include these fields in the authorization decision.

## 13.5 Standardize time zones and parsing

The code parses membership values without time-zone information and compares them with the server’s local time. SQLite datetime("now") defaults to UTC, while localtime() uses the host’s local zone. If membership timestamps are stored in a different time basis, comparisons can be wrong. Store timestamps consistently (preferably UTC) and use datetime objects with a defined time-zone policy.

## 13.6 Read current time once

localtime(time()) is called twice in the same condition. Capture one now value and compare start <= now <= expiry to make the decision easier to understand and prevent values being taken at slightly different instants.

## 13.7 Return a consistent response contract

The endpoint returns the strings admin, private, True, and False. If clients interpret these responses, document the meanings precisely. A JSON response such as {"decision": "allowed", "reason": "active_membership"} would be easier to extend, but changing from plain text requires updating test.py and any consuming client.

## 13.8 Select only the needed user columns

SELECT * retrieves password_hash and other user fields even though the endpoint only needs the user id, role, and possibly status. Prefer selecting only those columns; this reduces unnecessary exposure of sensitive account data inside application code.

### 13.9 Add a real access event

If this endpoint is intended to be part of the final ANPR access decision, consider recording each request in access_events with gate_id, normalized/read plate, matched vehicle id if found, confidence if available, decision, reason, and timestamp. Do so only after the CV-to-backend contract and required gate identifier are defined. Use audit_logs for administrative/accountability actions as appropriate.

### 13.10 Enforce foreign keys on the application connection

The direct SQLite inspection connection reported PRAGMA foreign_keys=0. SQLite foreign-key enforcement is configured per connection. Verify that the connection used by the application enforces foreign keys if you rely on those REFERENCES constraints to prevent orphaned data.

### 13.11 Remove unused import

strftime is imported but never used in the current code. Removing unused imports keeps the implementation clear.

## 14. Suggested Validation Checklist

- A known plate belonging to an admin returns the text admin.
- A known plate belonging to a private user returns the text private.
- A member with a valid, active membership returns the intended positive response.
- A plate that is not registered returns False.
- A registered member without memberships is handled without raising an exception.
- Cancelled and expired memberships are denied if that is the intended policy.
- Blocked vehicles and suspended/deleted users are handled according to policy.
- Plates with spaces and punctuation normalize to the same format stored in plate_normalized.
- The server and database use the same timezone convention for membership boundaries.
- The client handles role strings and boolean-like text responses consistently.

## 15. Conclusion

The current project has progressed from a small Flask/Fiftyville check into an ANPR backend built with FastAPI and a relational SQLite database. The updated code connects a normalized number plate to a registered vehicle owner, identifies the owner’s role, and checks a membership time interval for ordinary members.

The new database is broader than the current endpoint: it also defines plans, payments, garages, gates, parking sessions, access events, gate commands, audit logs, and notifications. These structures provide a foundation for the rest of the application, but the current code does not yet use them to record an access decision or operate a gate.

The most important next improvements are to handle missing/multiple memberships safely, validate membership/user/vehicle statuses, standardize timestamp time zones, select only required user columns, and agree on a consistent API response shape with the separate CV client.

# Appendix A. Current Code Reference

The following is the current implementation supplied for this report, formatted for readability. Its logic is reproduced without changing the API behavior:

```
from time import strftime, strptime, localtime, time
from cs50 import SQL
from fastapi import FastAPI
from fastapi.responses import PlainTextResponse
from pydantic import BaseModel

app = FastAPI()
db = SQL("sqlite:///anpr.db")

class PlateRequest(BaseModel):
    plate: str

@app.get("/", response_class=PlainTextResponse)
def index():
    return "Todo"

@app.post("/", response_class=PlainTextResponse)
def checking(data: PlateRequest):
    data.plate = normalized(data.plate)
    check = db.execute(
        "SELECT * FROM users WHERE id = "
        "(SELECT owner_user_id AS n FROM vehicles WHERE plate_normalized = ?)",
        data.plate
    )

    if check:
        if check[0]["role"] in ["admin", "private"]:
            return check[0]["role"]

        member = db.execute(
            "SELECT * FROM memberships WHERE user_id = ?",
            check[0]["id"]
        )
        member[0]["starts_at"] = strptime(
            member[0]["starts_at"], "%Y-%m-%d %H:%M:%S"
        )
        member[0]["expires_at"] = strptime(
            member[0]["expires_at"], "%Y-%m-%d %H:%M:%S"
        )

        if (member[0]["starts_at"] <= localtime(time()) and
                localtime(time()) <= member[0]["expires_at"]):
            return "True"
        return "False"

    return "False"

def normalized(plate: str) -> str:
    word = ""
    for i in plate:
        if i.isalnum():
            word += i
    return word.upper()
```